

Q.No:8

Question description

You are given a tree consisting of n nodes. A *matching* is a set of edges where each node is an endpoint of at most one edge. What is the maximum number of edges in a matching?

Input Format

The first input line contains an integer n : the number of nodes. The nodes are numbered $1, 2, \dots, n$.

Then there are $n-1$ lines describing the edges. Each line contains two integers a and b : there is an edge between nodes a and b .

Output Format

Print one integer: the maximum number of pairs.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq a, b \leq n$

Example

Input:

```
5
1 2
1 3
3 4
3 5
```

Output:

```
2
```

Explanation: One possible matching is $\{1, 2\}$ and $\{3, 4\}$

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5
1 2
1 3
3 4
3 5
```

EXPECTED OUTPUT

```
2
```

Test Case 2

INPUT (STDIN)

```
5
1 1
1 2
2 3
3 4
```

EXPECTED OUTPUT

```
6
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void dfs(int p,int i)
```

Test Case 2

KEYWORD

```
void link(int i,int j)
```

Code:

```
#include <stdio.h>
#include <string.h>

#define MAX 100005
```

```
#define MAXE 200005

int head[MAX];
int to[MAXE];
int next[MAXE];
int edgeCount = 0;

int dp0[MAX];
int dp1[MAX];

void link(int i, int j)
{
    to[edgeCount] = j;
    next[edgeCount] = head[i];
    head[i] = edgeCount++;
}

void dfs(int p, int i)
{
    int e, child;

    dp0[i] = 0;
    for (e = head[i]; e != -1; e = next[e])
    {
        child = to[e];

        if (child == p)
            continue;

        dfs(i, child);

        if (dp0[child] > dp1[child])
            dp0[i] += dp0[child];
        else
            dp0[i] += dp1[child];
    }

    dp1[i] = dp0[i];
    for (e = head[i]; e != -1; e = next[e])
    {
        child = to[e];

        if (child == p)
            continue;

        int currentMax;

        if (dp0[child] > dp1[child])
            currentMax = dp0[child];
        else
            currentMax = dp1[child];

        int value = dp0[i] - currentMax + dp0[child] + 1;
    }
}
```

```
if (value > dp1[i])
dp1[i] = value;
}
}
int main()
{
int n;
int i, a, b;
scanf("%d", &n);

for (i = 1; i <= n; i++)
{
head[i] = -1;
}
printf("\nOutput:\n");
for (i = 0; i < n - 1; i++)
{
scanf("%d %d", &a, &b);

link(a, b);
link(b, a);
}

dfs(0, 1);

if (dp0[1] > dp1[1])
printf("%d\n", dp0[1]);
else
printf("%d\n", dp1[1]);

return 0;
}
```

Output:

Test_case 1:

```
5
1 2
1 3
3 4
3 5
Output:
2
o aadityajaiswal@Aadity
```

Test_case 2:

```
adityajaisw
5
1 2
1 3
3 4
3 5
Output:
6
```